

보안관제 프로젝트 시스템 아키텍처 및 기본설계서(HLD) v1.0

보안관제 프로젝트 시스템 아키텍처 및 기본설계서 (HLD) v1.0

Project: SOC Detection & Monitoring Lab

Document Type: High-Level Design

Version: 1.0

Baseline Date: 2026-08-24

Status: Architecture Baseline Candidate

1. 문서 개요

1.1 목적

본 문서는 선행 산출물인 **「보안관제 포트폴리오 프로젝트 요구사항 정의서 v1.0」**에서 정의된 요구사항을 실제 구축 가능한 시스템 구조로 변환하기 위한 **High-Level Design(HLD)** 문서이다.

본 단계에서는 구체적인 설치 명령, IP 주소, Configuration Parameter까지 확정하지 않는다.

대신 다음 항목을 Architecture Baseline으로 확정한다.

- Deployment Model
- Network Zone
- Network Topology
- IDS Sensor 위치
- Traffic Capture 방식
- Suricata / Snort 역할
- Wazuh 배치
- Log Pipeline
- Security Boundary

- Storage 및 Evidence 구조
- Component 간 통신관계
- Failure Domain
- Portfolio Demonstration Architecture

2. 설계 목적

본 Architecture는 다음 질문에 답하는 것을 목표로 한다.

Attacker에서 발생한 Traffic이 어떤 경로로 Victim에 도달하고, Suricata가 그 Traffic을 어떻게 관찰하며, 생성된 `eve.json` Event가 Wazuh까지 어떻게 전달되고, SOC Analyst가 Raw Event·PCAP·Detection Rule을 근거로 True/False Positive를 판단할 수 있는가?

최종적으로 다음 End-to-End Flow가 Architecture 차원에서 성립하여야 한다.

```

Attack
↓
Network Traffic
↓
Victim Boundary
↓
Port Mirroring
↓
Suricata Sensor
↓
Detection Rule
↓
EVE JSON
↓
Wazuh Agent
↓
Wazuh Server
↓
Wazuh Indexer
↓
Wazuh Dashboard
↓
SOC Analyst
↓
Investigation
↓
Verdict
↓
Response
↓
Detection Tuning
↓

```

Re-test
↓
Portfolio Evidence

3. Requirements Baseline

본 HLD에서는 다음 Baseline을 변경하지 않는다.

항목	Baseline
Project	SOC Detection & Monitoring Lab
Primary Objective	보안관제 직무 지원용 실무 Portfolio
Primary IDS	Suricata 8.x
Secondary IDS	Snort 3.x
Primary SIEM	Wazuh
Packet Analysis	Wireshark / tshark / PCAP
Threat Framework	MITRE ATT&CK
Primary Flow	Traffic → Detection → Event → SIEM → Alert → Investigation → Response → Tuning → Evidence

Priority

P0

- IDS
- Traffic
- Rule
- Raw Log
- PCAP
- Analysis
- Evidence

P1

- Wazuh
- Dashboard

- MITRE ATT&CK
- False Positive
- Detection Tuning
- Incident Report

P2

- Zeek
- Sigma
- YARA
- Threat Intelligence
- Detection-as-Code
- Automation

4. Architecture 설계원칙

4.1 Portfolio First

Architecture의 우선순위는 제품의 개수가 아니라 **분석과정을 얼마나 증명할 수 있는가**이다.

```
도구 설치
<
탐지 성공
<
Raw Event 분석
<
PCAP 검증
<
판정 근거 설명
<
False Positive 분석
<
Rule Tuning
<
Re-test 및 Evidence
```

4.2 Passive IDS 우선

MVP에서는 Suricata를 Inline IPS가 아니라 **Passive IDS Sensor**로 사용한다.

목적은 차단보다 다음 능력을 확보하는 데 있다.

- Traffic Capture
- Signature 분석
- Protocol 분석
- Event 분석
- Packet 분석
- Alert Triage

Suricata 8.0.6 공식 문서는 Passive IDS, Active IPS, Offline PCAP을 모두 Tier 1 운영 방식으로 지원하며 AF_PACKET 역시 Tier 1 Capture 방식으로 분류한다.

4.3 Network Traffic 가시성 우선

IDS Sensor가 단순히 자신에게 들어오는 Traffic만 관찰하는 구조를 피한다.

실제 SOC/NIDS의 SPAN Port와 유사하게

| **Virtual Switch → Mirrored Traffic → Security Sensor**

구조를 사용한다.

4.4 공격망 격리

Attack Traffic은 Public Network로 전달되지 않는다.

Attack/Victim Network에는 기본적으로 직접적인 Internet Gateway를 제공하지 않는다.

4.5 Management Plane 분리

IDS 분석 Traffic과 관리 Traffic을 논리적으로 분리한다.

Data Plane
≠
Management Plane

4.6 Source of Truth 유지

SIEM 데이터만 원본으로 취급하지 않는다.

각 데이터의 원본을 별도로 유지한다.

```
Network      → PCAP
Suricata     → eve.json
Snort        → Raw Alert
Detection Rule → Git
SIEM Alert   → Wazuh
Analysis     → Incident Report
```

5. Architecture 대안 비교

5.1 Option A — Docker 중심

```
Windows
  ↓
WSL2 / Docker
├─ Victim
├─ IDS
├─ SIEM
└─ Tools
```

장점

- 구축이 간단함
- 빠른 재생성
- GitHub 재현성이 높음
- Resource 소비가 상대적으로 적음

단점

- Network Interface 개념이 추상화됨
- IDS가 Traffic을 어디에서 관찰하는지 설명하기 어려움
- Docker Bridge/NAT가 Network Security 학습을 방해할 수 있음
- 실제 SPAN/TAP Sensor 구조와 차이가 큼

6. Option B — VM 중심

```
Windows
├─ Attacker VM
```

```

├ Victim VM
├ Sensor VM
└ SIEM VM

```

장점

- 실제 Network 구조와 유사
- Network Interface가 명확
- Packet Capture 학습 효과가 큼
- 높은 격리성

단점

- RAM 및 Storage 사용량 증가
- 재현성이 Docker보다 낮음
- Snapshot/VM 관리 부담 증가

7. Option C — Hybrid

```

Windows Host
├ Hyper-V
│   ├── Attacker VM
│   ├── Gateway VM
│   ├── Victim VM
│   └ Security Sensor VM
└ Docker Desktop / WSL2
    └ Wazuh Single-node

```

7.1 비교

기준	Docker	VM	Hybrid
Network 현실성	2/5	5/5	5/5
Packet Capture	2/5	5/5	5/5
IDS 학습 가치	3/5	5/5	5/5
Network 격리	3/5	5/5	5/5
구축 난이도	5/5	2/5	3/5

기준	Docker	VM	Hybrid
Resource 효율	5/5	2/5	3/5
재현성	5/5	3/5	4/5
Troubleshooting	3/5	4/5	5/5
Portfolio 가치	3/5	5/5	5/5
확장성	4/5	4/5	5/5

8. 최종 Architecture 선정

결정

| Option C — Hybrid Architecture 채택

구체적으로:

```
Windows Host
+
Hyper-V Virtual Network
+
Linux VM
+
Hyper-V Port Mirroring
+
Suricata
+
Snort
+
Docker Desktop / WSL2
+
Wazuh Single-node
```

를 사용한다.

8.1 선정 이유

이유 1 — Network 구조 확인 가능

Attacker, Gateway, Victim, Sensor를 별도 VM으로 분리하여 Network Interface와 Traffic Path를 직접 확인할 수 있다.

이유 2 — SPAN 개념 재현

Hyper-V는 VM Network Adapter를 **Source** 와 **Destination** Port Mirroring으로 설정하여 Source VM의 송수신 Packet을 Monitoring VM으로 복제하는 기능을 제공한다. Source와 Destination NIC는 동일 Virtual Switch에 연결되어야 한다.

따라서 물리환경의

Switch
→ SPAN Port
→ IDS Sensor

개념을

Hyper-V vSwitch
→ Port Mirroring
→ Suricata Sensor VM

형태로 재현할 수 있다.

이유 3 — SIEM 재현성

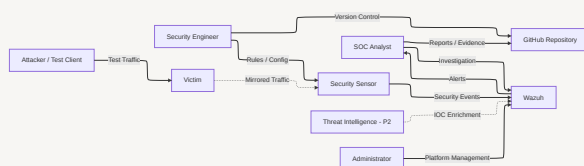
Wazuh는 Docker에서 공식적으로 single-node deployment를 지원하고 Wazuh Server, Indexer, Dashboard를 각각 Container로 구성한다.

이유 4 — Failure Domain 분리

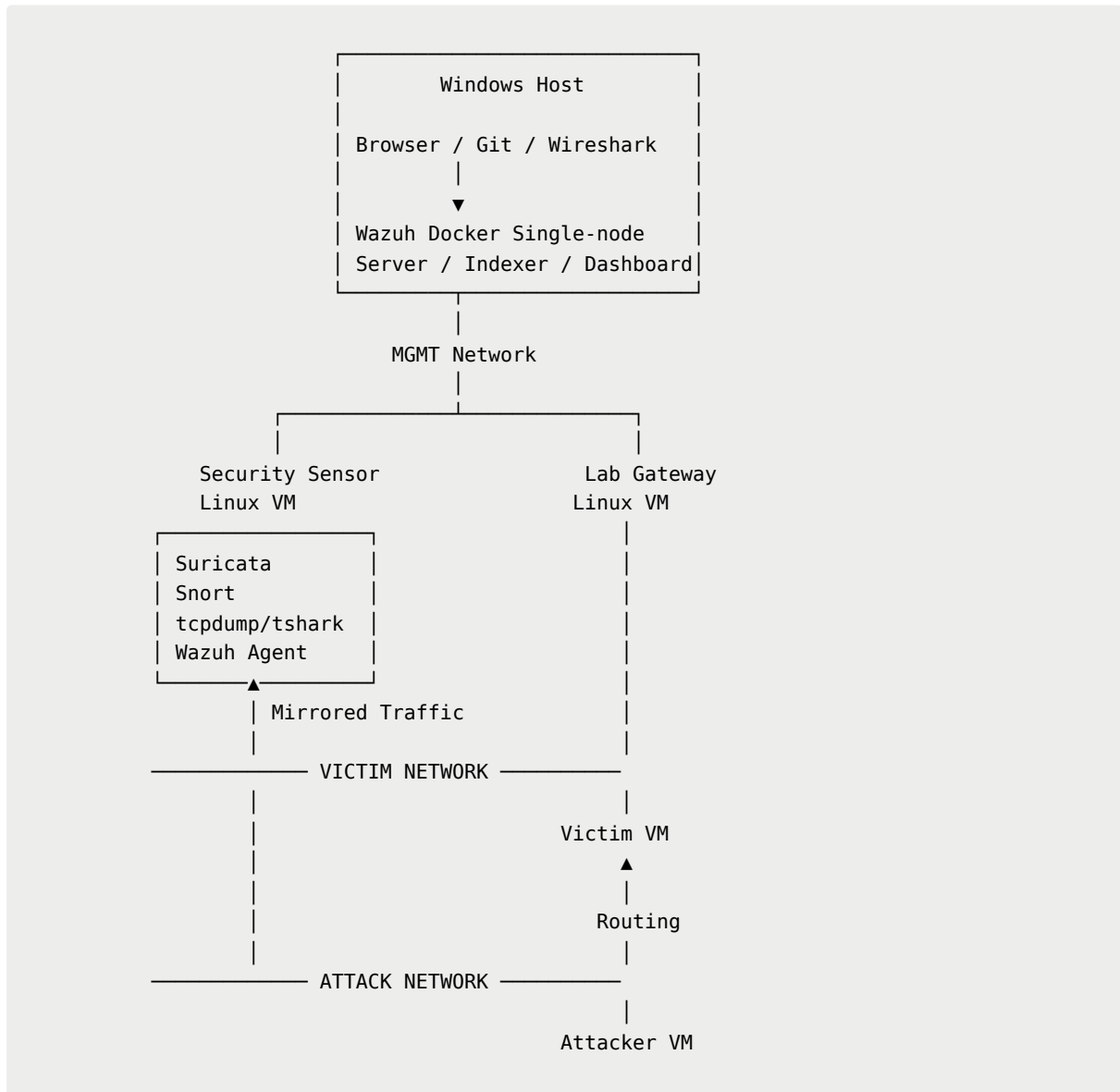
IDS와 SIEM을 분리하기 때문에 다음 문제를 개별 분석할 수 있다.

Traffic 문제
IDS 문제
Rule 문제
Log 문제
Agent 문제
Wazuh 문제
Dashboard 문제

9. System Context



10. Overall Architecture



11. Component Architecture

ID	Component	Runtime	역할	Priority
CMP-001	Attacker	Linux VM	Test Traffic 발생	P0
CMP-002	Victim	Linux VM	분석 대상	P0
CMP-003	Suricata	Sensor VM	Primary IDS	P0
CMP-004	Snort	Sensor VM	Secondary Validation	P0

ID	Component	Runtime	역할	Priority
CMP-005	Packet Capture	Sensor VM	PCAP Evidence	P0
CMP-006	Wazuh Agent	Sensor/Victim	Log 수집	P1
CMP-007	Wazuh Server	Container	Event 분석	P1
CMP-008	Wazuh Indexer	Container	Index/Search	P1
CMP-009	Wazuh Dashboard	Container	SOC Interface	P1
CMP-010	Analyst Workstation	Windows	Investigation	P0
CMP-011	Evidence Storage	Host/Git	증빙 관리	P0
CMP-012	Git Repository	GitHub	Version Control	P0
CMP-013	Lab Gateway	Linux VM	Zone Routing/Isolation	P0

12. Component Responsibility

CMP-001 Attacker

Purpose

공격·이상 Traffic을 생성하는 Lab Actor.

Input

Attack Scenario

Output

Network Traffic

Zone

ZONE-ATTACK

Failure Impact

Attack Scenario 실행 불가.

CMP-002 Victim

Purpose

Network 및 Application 공격 Simulation 대상.

주요 서비스 후보

- HTTP
- SSH
- DNS Client
- 테스트 Application

Zone

ZONE-VICTIM

CMP-003 Suricata Sensor

Role

Primary Real-time IDS

Input

Mirrored Packet

Output

- EVE JSON
- Alert
- Protocol Event
- Stats

Suricata의 EVE는 Alert 및 Event를 JSON 형태로 출력하며 Logstash 등 외부 Tool과 연동하기 위한 형식으로 제공된다.

CMP-004 Snort

Role

Secondary IDS / Validation Engine

기본 역할:

```
PCAP
→ Snort
→ Detection
→ Suricata 결과와 비교
```

CMP-005 Packet Capture

도구:

- tcpdump
- tshark
- Wireshark

역할:

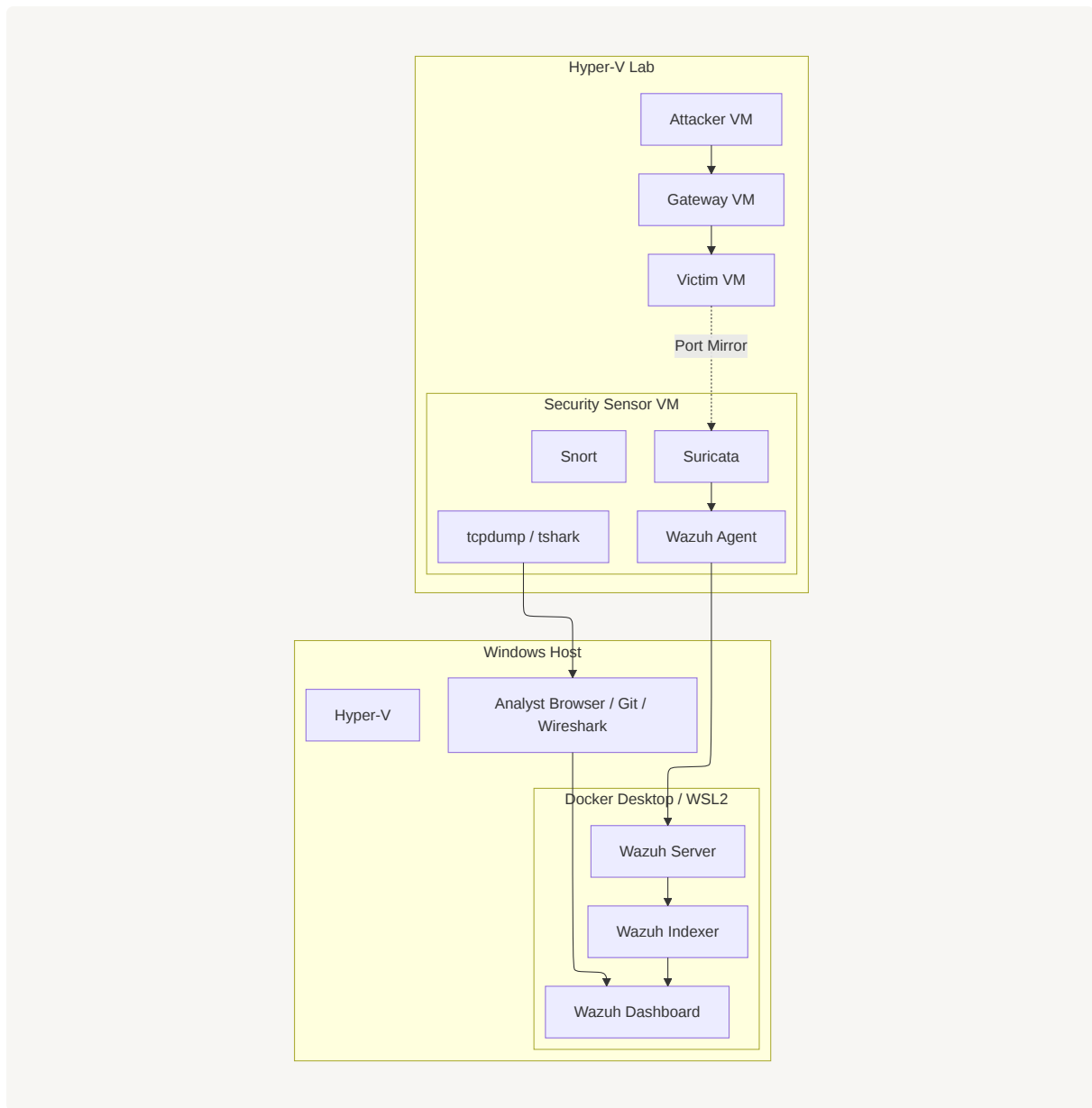
```
IDS Alert
→ Related PCAP
→ Packet Validation
```

CMP-006 Wazuh Agent

Sensor의 `eve.json` 을 수집하여 Wazuh Server로 전송한다.

Wazuh 공식 Suricata Integration 역시 Wazuh Agent가 `/var/log/suricata/eve.json` 을 JSON Log로 읽도록 구성하며, 이후 Wazuh가 해당 Event를 분석한다.

13. Deployment Architecture



14. WSL2 사용 원칙

WSL2는 IDS Data Plane으로 사용하지 않는다.

기본 WSL Networking은 NAT 구조이고 Microsoft도 이를 기본 Networking Mode로 설명한다. Mirrored Networking은 Windows 11 22H2 이상에서 제공된다.

따라서 HLD에서는 WSL Networking의 세부동작에 IDS Packet Capture를 의존하지 않는다.

WSL2의 역할은:

Docker Runtime
Wazuh Containers

로 제한한다.

Docker 역시 Windows 환경에서 WSL2 Backend를 공식 지원하며 WSL 2.1.5 이상을 최소 요구한다.

15. Network Zone

Persistent Network는 3개를 사용한다.

ZONE-MGMT

Purpose

Management / SOC 통신

예정 CIDR

10.77.10.0/24

Members

- Windows Host
- Security Sensor Management NIC
- Lab Gateway Management NIC
- Wazuh Published Service

ZONE-ATTACK

Purpose

공격자 Traffic 발생

예정 CIDR

10.77.20.0/24

Members

- Attacker
- Gateway Attack Interface

Internet

기본 차단.

ZONE-VICTIM

Purpose

Victim 및 Sensor Monitoring

예정 CIDR

10.77.30.0/24

Members

- Victim
- Gateway Victim Interface
- Sensor Monitoring NIC

16. Optional Maintenance Network

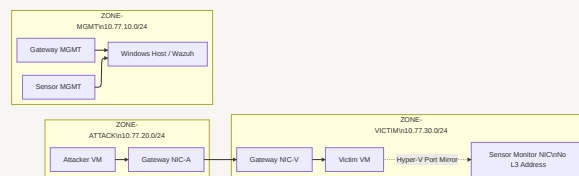
Package Update가 필요한 경우 일시적으로 Internet Access용 Maintenance NIC를 연결할 수 있다.

단:

Attack Simulation 실행 전 반드시 제거 또는 Disable한다.

Persistent Data Plane에는 포함하지 않는다.

17. Network Topology



18. IP 설계 원칙

HLD에서는 Subnet만 Baseline으로 확정한다.

실제 다음 값은 LLD에서 결정한다.

- Host IP
- Gateway IP

- Attacker IP
- Victim IP
- Sensor IP

원칙

- Static Address 사용
- Sensor Monitoring NIC에는 IP 미할당
- Attack Zone에는 Internet Default Gateway 미설정
- Victim Zone에서도 직접 Internet Route 금지
- Management Route는 Gateway Policy에 의해 제한

19. Traffic Path

19.1 공격 Traffic

```

Attacker
  ↓
Attack vSwitch
  ↓
Lab Gateway
  ↓
Victim vSwitch
  ↓
Victim
  
```

19.2 IDS Observation

Victim vNIC 또는 Victim Boundary Traffic을 Hyper-V Port Mirroring Source로 설정한다.

Sensor의 Monitoring vNIC를 Destination으로 설정한다.

```

Victim Traffic
  |
  |→ Normal Network Processing
  |
  |→ Mirrored Copy
      ↓
    Sensor NIC
  
```

↓
Suricata

Microsoft의 Hyper-V Port Mirroring에서는 Source Adapter의 송수신 Traffic이 Destination Adapter로 복사된다.

20. IDS Capture Architecture

Primary Capture

Hyper-V Virtual Switch Port Mirroring + Linux AF_PACKET

```
Hyper-V vSwitch
  ↓
Mirrored Packet
  ↓
Sensor Monitoring NIC
  ↓
AF_PACKET
  ↓
Suricata
```

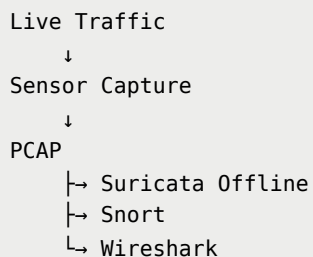
AF_PACKET은 Suricata 8.0.6 공식 Support Matrix에서 Tier 1 Capture 방식이다.

21. Capture 방식 비교

방식	실무 유사성	Lab 구현성	복잡도	선정
Physical TAP	5	1	5	X
Physical SPAN	5	2	4	X
Hyper-V Port Mirror	5	5	3	Primary
Inline Bridge	5	3	5	P2
NFQUEUE	4	3	5	P2
Docker Capture	2	5	2	X
PCAP Offline	3	5	1	Validation

22. Offline Validation

Offline 분석의 Source of Truth는 PCAP이다.

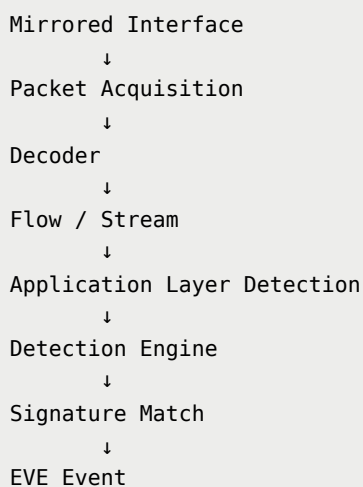


Suricata 공식 Support Matrix 역시 Offline PCAP을 Tier 1 Operation Mode로 지원한다.

23. Suricata Architecture

역할

Primary Real-time IDS



23.1 기본 정책

Monitoring Mode

Passive IDS

Packet Acquisition

AF_PACKET

HOME_NET

Victim Network를 중심으로 정의한다.

예:

```
HOME_NET
=
Victim Zone
```

실제 값은 LLD에서 확정한다.

23.2 Rule 구조

```
suricata/
├─ rules/
│   ├── community/
│   ├── emerging-threats/
│   ├── custom/
│   └─ lab/
```

23.3 Output

Primary:

```
eve.json
```

Secondary:

```
fast.log
```

`eve.json` 을 SIEM Integration의 Source로 사용한다.

24. EVE Event Model

최소 다음 Event를 활용한다.

- alert

- flow
- http
- dns
- tls
- stats

Suricata EVE는 Alert/Event를 JSON으로 출력할 수 있다.

25. Snort Architecture

역할

| Secondary Validation / Comparative IDS

Snort는 기본적으로 Real-time Primary Sensor로 사용하지 않는다.

```

PCAP
↓
Snort 3
↓
Inspector
↓
Detection Engine
↓
Rule Match
↓
Raw Alert
↓
Suricata 결과 비교

```

현재 Snort 공식 사이트에서 Snort 3.12.2.0 Source와 관련 Package를 제공하고 있다.

26. Suricata / Snort 역할 분리

항목	Suricata	Snort
역할	Primary IDS	Secondary IDS
실시간 Capture	기본	Optional
Offline PCAP	사용	중점 사용
SIEM Integration	Primary	Optional

항목	Suricata	Snort
Rule 학습	O	O
Detection 비교	기준	비교대상
Evidence	EVE + PCAP	Alert + 비교표

설계 이유

양쪽 IDS를 항상 실시간으로 동시에 운영하면

- CPU 증가
- Duplicate Alert
- Rule Source 차이
- 분석 복잡도 증가

가 발생할 수 있다.

따라서 Portfolio에서는

```

Suricata
=
운영 IDS

Snort
=
비교 / 검증 IDS

```

로 구분한다.

27. Packet Analysis Architecture

```

Alert
↓
Event Timestamp
↓
Related PCAP
↓
Wireshark / tshark
↓
5-Tuple
↓
TCP Stream / Payload
↓
Detection Rule

```

↓
Verdict

PCAP는 단순 Screenshot보다 높은 가치의 Evidence로 간주한다.

28. Wazuh Architecture

Wazuh는 **Docker Single-node**로 배치한다.

Wazuh 공식 Docker Architecture는 Single-node에서 각각 별도의

- Wazuh Indexer Container
- Wazuh Manager Container
- Wazuh Dashboard Container

를 사용한다.

```
Wazuh Agent
  ↓
Wazuh Server
  ↓
Analysis
  ↓
Wazuh Indexer
  ↓
Wazuh Dashboard
  ↓
SOC Analyst
```

29. Wazuh 배치 결정

Deployment

```
Windows Host
  ↓
Docker Desktop
  ↓
WSL2 Backend
  ↓
Wazuh Single-node Containers
```

선택 이유

- 공식 Docker Deployment 제공
- 재현성
- Wazuh Component 분리 학습
- Snapshot 대신 Volume Backup 가능
- GitHub Portfolio 구성에 유리

30. Wazuh Resource 기준

Wazuh 공식 Docker single-node 요구사항은 다음과 같다.

Resource	Official Minimum
CPU	4 cores
RAM	8 GB
Disk	50 GB

또한 Linux/WSL2 Docker Host에서는 Indexer를 위해 `vm.max_map_count` 262144 이상을 요구한다.

따라서 Wazuh를 전체 Lab에서 가장 Resource-intensive한 Component로 간주한다.

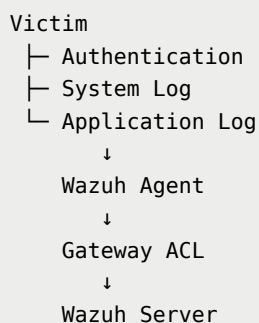
31. Suricata → Wazuh Log Pipeline

```
Mirrored Traffic
  ↓
Suricata
  ↓
eve.json
  ↓
Wazuh Agent
  ↓
Wazuh Server
  ↓
Decoder / Rule
  ↓
Indexer
  ↓
Dashboard
```


Wazuh 공식 Network IDS Integration에서도 Agent가 Suricata `eve.json` 파일을 JSON 형식으로 수집하고 관련 Alert를 Dashboard에 생성하는 구조를 제시한다.

32. Victim Endpoint Log Pipeline

P1에서 Victim에 Wazuh Agent를 추가한다.



Attack Zone에서는 Wazuh Management Network로 직접 연결할 수 없도록 한다.

33. Log Source of Truth

Data	Source of Truth
Network Packet	PCAP
Suricata Detection	<code>eve.json</code>
Suricata Simple Alert	<code>fast.log</code>
Snort Detection	Snort Raw Alert
Endpoint Log	Victim Original Log
SIEM Alert	Wazuh
Detection Rule	Git
ATT&CK Mapping	Incident/Rule Metadata
Analyst Decision	Incident Report
Screenshot	Evidence Repository

34. Storage Architecture

Storage를 4계층으로 구분한다.

Layer 1 — Raw

logs/raw/
pcaps/raw/

Layer 2 — Security Platform

Wazuh Indexer
Docker Volume

Layer 3 — Evidence

evidence/

Layer 4 — Documentation

docs/

35. Retention 정책

본 정책은 **Lab Recommendation**이며 규제상 보존기간이 아니다.

Data	기본 Retention
Raw Suricata Log	30일
Wazuh Indexed Alert	최대 90일 목표
일반 PCAP	14일
주요 Scenario PCAP	영구 보관
Screenshot	영구
Incident Report	영구
Detection Rules	Git History
Test Result	Project Life

Wazuh 공식 Quickstart도 1~25 Agent 환경에서 90일 Queryable/Indexed Alert 저장을 기준으로 50GB 수준을 예시로 제시한다.

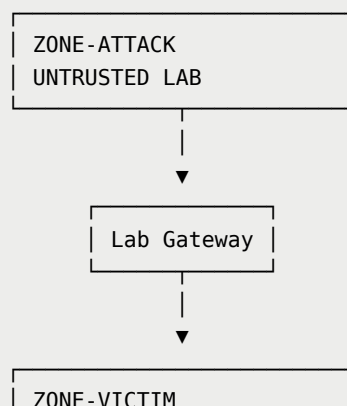
36. Evidence Architecture

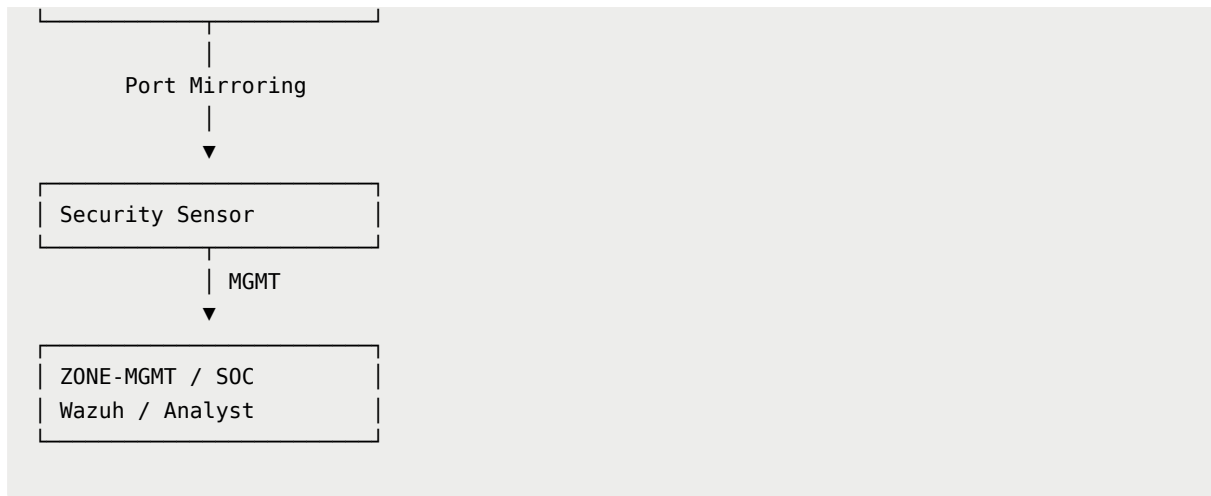
```
evidence/  
├─ EV-NET/  
├─ EV-SURI/  
├─ EV-SNORT/  
├─ EV-WAZUH/  
├─ EV-ATTACK/  
├─ EV-ANALYSIS/  
├─ EV-TUNE/  
└─ EV-TROUBLE/
```

각 Evidence에는 다음 Metadata를 연결한다.

```
Evidence ID  
Requirement ID  
Test Case  
Timestamp  
Component  
Scenario  
Result  
Screenshot  
Raw Log  
PCAP  
Rule  
Git Commit
```

37. Security Boundary





38. Security Policy

Attack → Management

DENY

Attack → Victim

ALLOW

Lab Scenario 범위 내.

Victim → Management

기본 Deny.

다음 P1 Log Transport만 제한적으로 허용 가능.

Victim Agent
→ Wazuh

Sensor → Management

Wazuh Event 전송 허용.

39. Management Plane / Data Plane

Data Plane

Attacker
→ Gateway
→ Victim
→ Mirror
→ Suricata

Management Plane

Sensor
→ Wazuh

Analyst
→ Dashboard

Engineer
→ Rule / Git

Administrator
→ Infrastructure

두 Plane을 동일 Interface로 합치지 않는 것을 원칙으로 한다.

40. Port / Protocol Matrix

공식 Wazuh Default Port 기준으로 작성한다.

Source	Destination	Protocol/Port	Purpose	Exposure
Wazuh Agent	Wazuh Server	TCP/1514	Agent Event	MGMT
Wazuh Agent	Wazuh Server	TCP/1515	Enrollment	MGMT
Analyst	Dashboard	TCP/443	HTTPS Dashboard	MGMT
Dashboard/Host	Server API	TCP/55000	API	Internal/MGMT
Dashboard/Server	Indexer	TCP/9200	Index API	Internal
Syslog Device	Wazuh	UDP/514	Optional Syslog	Disabled by default

Wazuh 공식 Docker 배포는 1514, 1515, 514, 55000, 9200, 443 포트를 정의한다.

Security Decision

9200 과 55000 은 Attack/Victim Zone에 공개하지 않는다.

41. Resource Sizing

41.1 VM Resource

아래 값은 별도 표기가 없는 경우 ****[Lab Recommendation]****이다.

Component	vCPU	RAM	Disk
Attacker	2	2~4 GB	30 GB
Gateway	1	1 GB	10 GB
Victim	2	2~4 GB	30 GB
Sensor	4	4~8 GB	50 GB
Wazuh Docker	4	8 GB	50 GB+

Wazuh 값은 공식 Docker 최소사양이다.

41.2 Host Recommendation

[Estimated Lab Recommendation]

CPU:
8 logical processors 이상 권장

RAM:
32 GB 권장

Storage:
200~300 GB 이상 여유공간 권장

16GB 환경에서는 모든 Component를 동시에 실행하기보다 Phase별 실행을 권장한다.

42. Failure Domain

FD-001 Attacker Down

영향:

Live Attack Simulation 불가

IDS/SIEM 자체 검증은 PCAP으로 가능.

FD-002 Gateway Down

Attack
x
Victim

Zone 간 Traffic 중단.

FD-003 Port Mirror Failure

Attacker
→ Victim 정상

Suricata
→ Traffic 미관찰

이는 중요한 Troubleshooting Scenario가 된다.

FD-004 Suricata Down

Network 정상
PCAP Capture 가능
IDS Alert 중단

FD-005 Wazuh Agent Down

Suricata
→ eve.json 생성 0

Wazuh
→ Ingestion X

따라서 IDS 정상 여부와 SIEM Pipeline 문제를 분리할 수 있다.

FD-006 Wazuh Down

Suricata Detection 0
Raw Event 0

FD-007 Indexer Down

Wazuh Manager는 Event를 처리할 수 있으나 정상 Search/Index 기능이 손상된다.

43. Health Check Architecture

각 계층에 별도 Health Check를 둔다.

Layer	Check
Network	Connectivity / Route
Mirror	Sensor Packet Count
Suricata	Process + Stats
EVE	최근 Event Timestamp
Wazuh Agent	Agent Status
Manager	Service Status
Indexer	Cluster Health
Dashboard	HTTPS Response
Storage	Free Disk
Git	Working Tree / Commit

실제 Command는 LLD에서 작성한다.

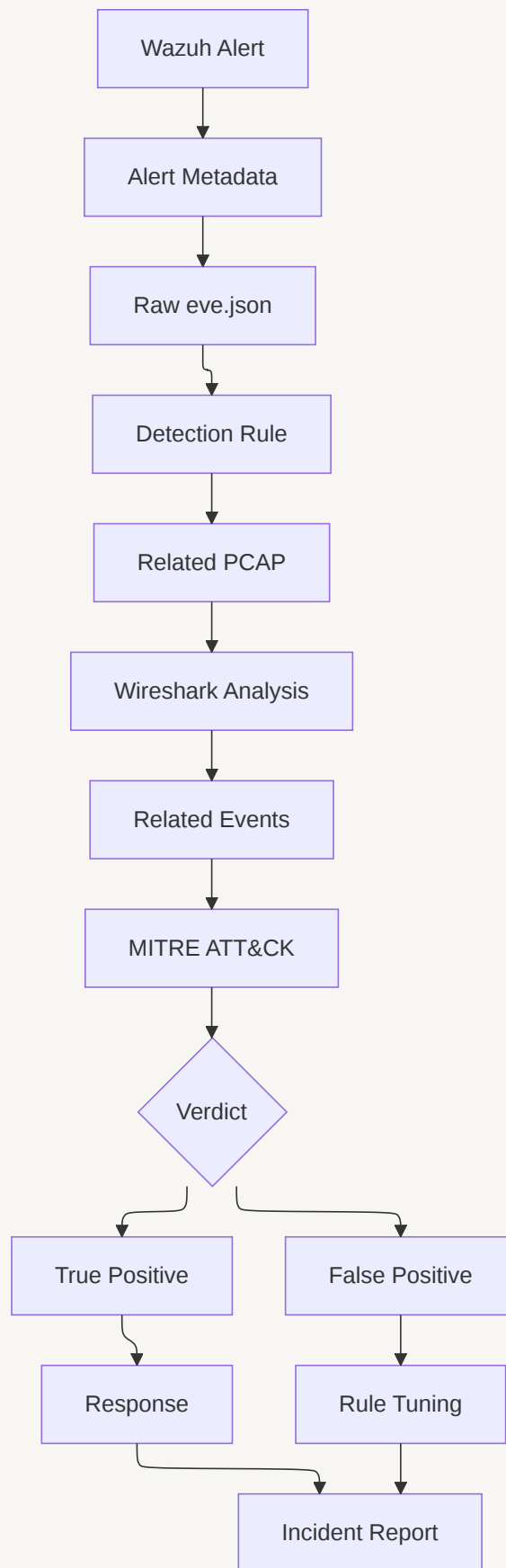
44. Attack Lab Architecture

Scenario	방식
ICMP	Live
SYN Scan	Live
Port Scan	Live
Enumeration	Live
SSH Login Failure	Live
Brute Force Simulation	Controlled Live
Suspicious HTTP	Synthetic Live

Scenario	방식
SQLi Pattern	Synthetic Live
XSS Pattern	Synthetic Live
Traversal Pattern	Synthetic Live
Suspicious DNS	Synthetic
Malware Traffic	PCAP Offline

실제 Malware 실행은 MVP 범위에서 제외한다.

45. SOC Investigation Architecture



46. Investigation Evidence Chain

Alert 하나에 다음 Evidence가 연결되어야 한다.

```
Alert ID
├─ EVE Event
├─ Rule SID
├─ PCAP
├─ Source/Destination
├─ Protocol
├─ ATT&CK
├─ Analyst Note
└─ Verdict
```

47. Detection Tuning Architecture

```
Alert
↓
Candidate FP
↓
Raw Event
↓
PCAP
↓
Rule Condition
↓
Root Cause
↓
Rule Modification
↓
Git Commit
↓
Re-test
↓
Attack Traffic Detection?
├─ YES
│   └─ Normal Traffic Alert Reduced?
│       ├── YES → VALIDATED
│       └─ NO  → TUNE AGAIN
```

48. Rule Lifecycle

DRAFT
↓
TEST
↓
VALIDATED
↓
DEPLOYED
↓
MONITORED
↓
TUNED
↓
DEPRECATED

49. MITRE ATT&CK Integration

ATT&CK Mapping은 3단계에서 적용한다.

Rule

SID
→ ATT&CK Technique

Alert

Security Event
→ ATT&CK Technique

Incident

Incident
→ Tactic
→ Technique
→ Evidence

2026년 8월 6일 MITRE ATT&CK 공식 Changelog에서 Content Version 19.2가 공개되었다.

50. Repository Architecture

```
soc-detection-monitoring-lab/
├── README.md
├── docs/
│   ├── 01-requirements/
│   ├── 02-architecture/
│   │   ├── HLD.md
│   │   ├── diagrams/
│   │   └── adr/
│   ├── 03-design/
│   ├── 04-deployment/
│   ├── 05-testing/
│   ├── 06-detection/
│   ├── 07-investigation/
│   ├── 08-incident/
│   └── 09-troubleshooting/
├── infrastructure/
│   ├── hyper-v/
│   └── docker/
├── suricata/
│   ├── config/
│   └── rules/
├── snort/
│   ├── config/
│   └── rules/
├── wazuh/
├── attack-lab/
├── pcaps/
│   └── samples/
├── scripts/
├── tests/
└── evidence/
```

51. Repository Directory 책임

Directory	역할
docs/01-requirements	SRS
docs/02-architecture	HLD / ADR
docs/03-design	LLD
docs/04-deployment	구축 절차
docs/05-testing	Test Case
docs/06-detection	Rule 분석
docs/07-investigation	SOC 분석
docs/08-incident	Incident Report
docs/09-troubleshooting	장애 기록
infrastructure	Infra-as-Code
suricata	Suricata 설정
snort	Snort 설정
wazuh	Wazuh 설정
pcaps	검증 Traffic
evidence	Portfolio Evidence

52. GitHub 공개 정책

공개

- Architecture Diagram
- Non-secret Configuration
- Custom Rule
- Test Scripts
- Sanitized Log
- Sanitized PCAP
- Incident Report
- Troubleshooting
- Detection Tuning 사례

비공개

- Password
 - Token
 - API Key
 - 개인 Public IP
 - 실제 Credential
 - 실제 민감정보
 - 공격 가능한 외부 Target 정보
 - 개인 식별정보
-

53. ADR-001 Deployment Model

Status

Accepted

Context

Packet Capture와 Portfolio 재현성을 동시에 확보해야 한다.

Options

- Docker
- VM
- Hybrid

Decision

Hybrid

Reason

VM 기반 Network 가시성과 Docker 기반 SIEM 재현성을 동시에 확보.

Risk

Resource 사용량 증가.

54. ADR-002 Primary IDS

Decision

Suricata 8.x

Reason

- Real-time NIDS
 - EVE JSON
 - Protocol Metadata
 - SIEM 연계
 - AF_PACKET
-

55. ADR-003 Secondary IDS

Decision

Snort 3.x

Role

PCAP Validation 및 Rule 비교.

56. ADR-004 Primary SIEM

Decision

Wazuh

Deployment

Docker Single-node

Reason

IDS Event + Endpoint Event + Dashboard를 동일 플랫폼에서 실습 가능.

57. ADR-005 Network Isolation

Decision

Hyper-V Private/Internal Virtual Switch 분리.

Persistent Zone:

- MGMT
- ATTACK

- VICTIM

58. ADR-006 IDS Capture Method

Decision

Hyper-V Port Mirroring

Secondary

Offline PCAP

59. ADR-007 PCAP Evidence

주요 Incident PCAP는 Portfolio Evidence로 보존한다.

일반 Capture는 Retention 후 삭제할 수 있다.

60. ADR-008 Log Source of Truth

Decision

```
Suricata → eve.json  
Snort → Raw Alert  
Network → PCAP  
Rules → Git  
Analysis → Markdown Report
```

SIEM Index는 파생 데이터로 취급한다.

61. ADR-009 VM / Container Boundary

Component	Boundary
Network Actor	VM
Sensor	VM
Gateway	VM
Wazuh	Container
Analyst	Host

62. Requirements Traceability

Requirement	Architecture
FR-IDS-001	CMP-003 / Port Mirror
FR-IDS-002	Suricata
FR-IDS-003	Snort
FR-IDS-004	PCAP
FR-RULE-001	Git Rule Repository
FR-SIEM-001	Wazuh Agent/Server
FR-SIEM-002	Indexer
FR-SIEM-003	Dashboard
FR-ANALYSIS-001	Investigation Flow
SEC-001	Credential Policy
SEC-005	Zone Firewall
SEC-011	Isolated Attack Network

63. Architecture Validation Matrix

ID	Test	Success Criteria
AV-001	Attacker → Victim	정상 통신
AV-002	Port Mirror	Sensor에서 동일 Packet 관찰
AV-003	Suricata	Rule Match 발생
AV-004	Snort	PCAP Detection 성공
AV-005	PCAP	Capture 파일 생성
AV-006	EVE	Event JSON 생성
AV-007	Wazuh	Event Ingestion
AV-008	Search	Alert 검색
AV-009	Dashboard	HTTPS Access
AV-010	Isolation	Attack → MGMT 차단
AV-011	Evidence	Alert/PCAP 연결
AV-012	Failure Domain	IDS/SIEM 장애 구분

64. MVP Architecture

MVP-0 — Network

```
Attacker
→ Gateway
→ Victim
```

MVP-1 — Visibility

```
Victim
→ Mirror
→ Sensor
```

MVP-2 — Detection

```
Traffic
→ Suricata
→ Rule
→ eve.json
```

MVP-3 — Validation

```
PCAP
├→ Suricata
├→ Snort
└→ Wireshark
```

MVP-4 — SIEM

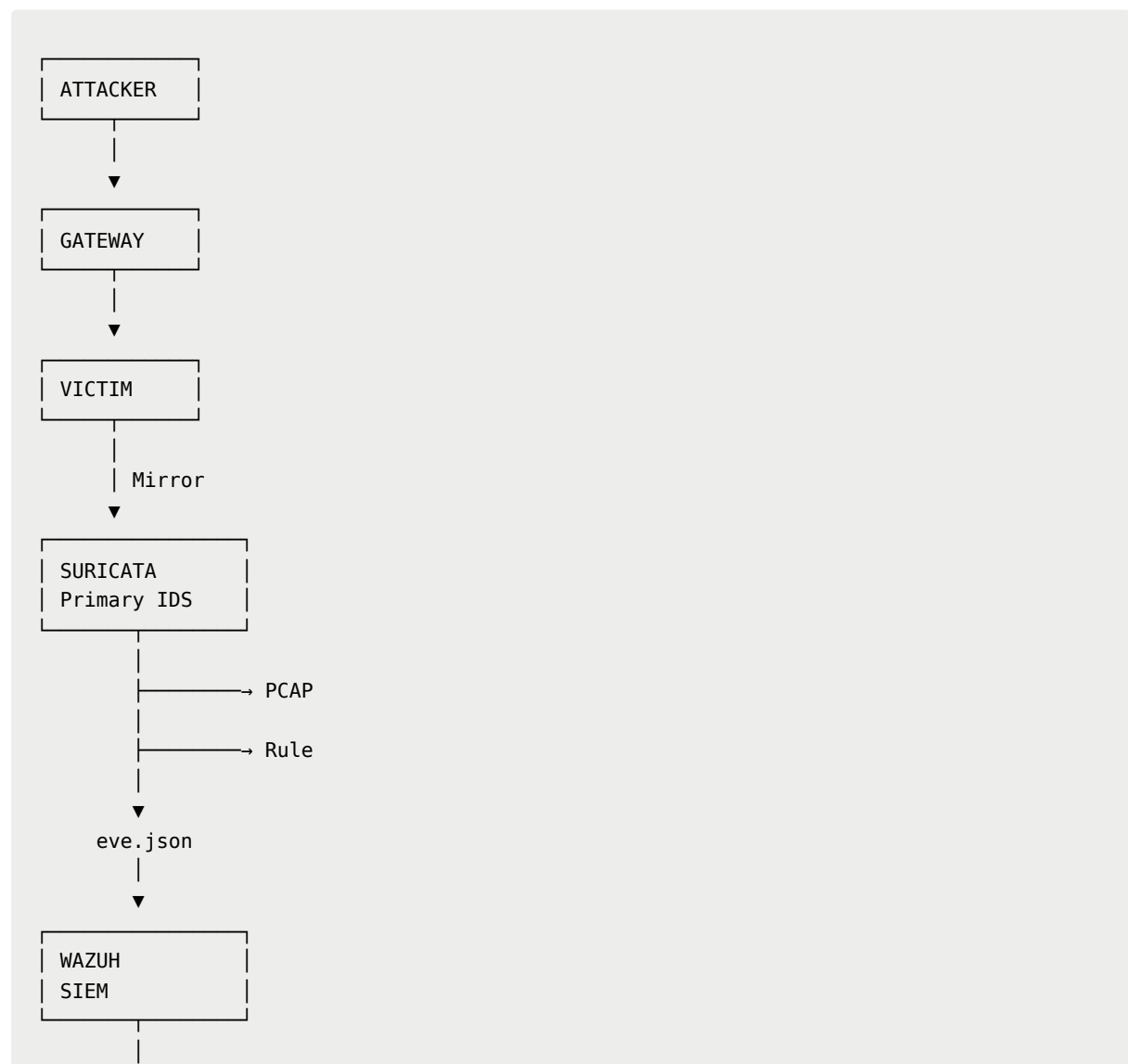
```
eve.json
→ Wazuh Agent
→ Server
→ Indexer
→ Dashboard
```

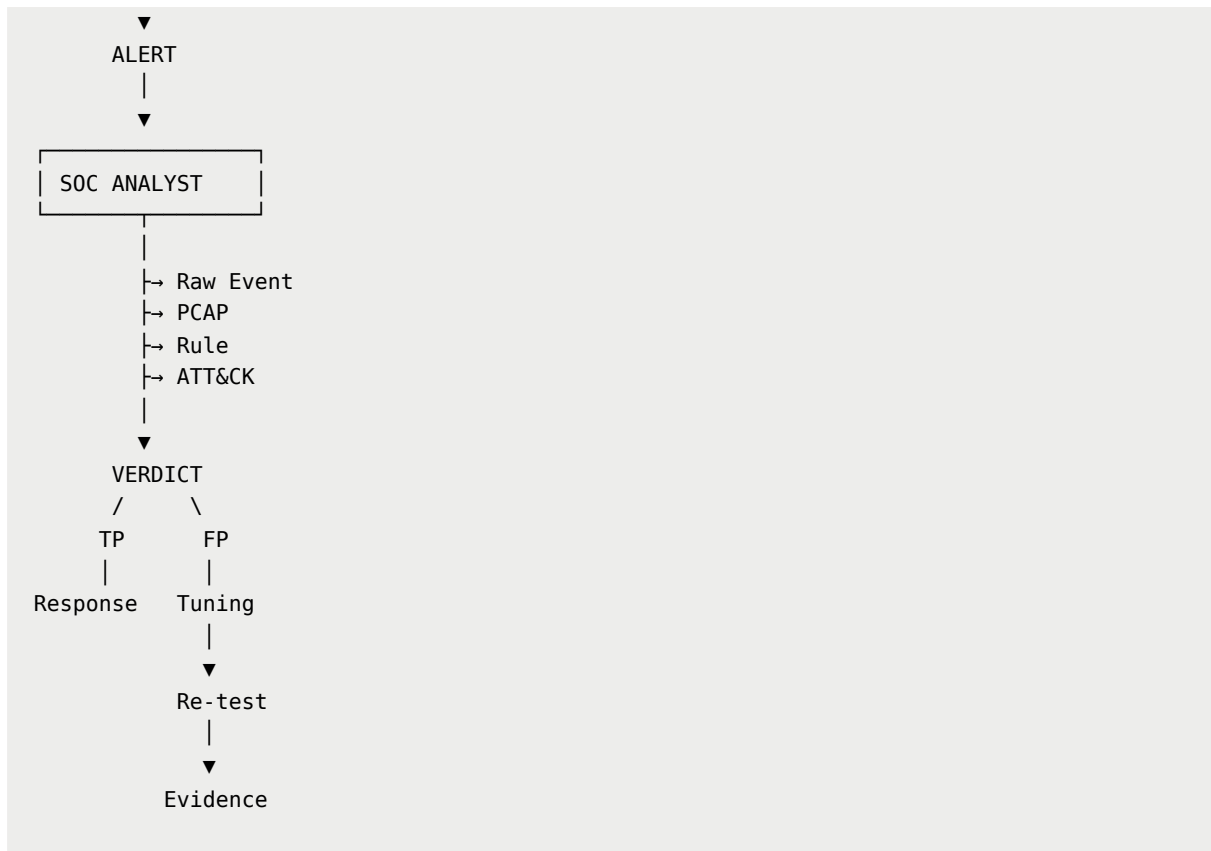
MVP-5 — SOC Workflow

Attack
→ Detection
→ Alert
→ Investigation
→ Verdict
→ Tuning
→ Re-test
→ Evidence

65. Portfolio Demonstration Architecture

면접에서 사용할 핵심 Diagram은 다음으로 고정한다.





66. Technology Verification

2026-08-24 기준

Technology	Verified Version	Status
Suricata	8.0.6	Current supported 8.x
Snort	3.12.2.0	Current Snort 3 download
Wazuh	4.14.7	Current Docker branch
OpenSearch	3.8	Current docs
Zeek	8.2	Current Feature Release
MITRE ATT&CK	19.2	Current content
Docker Desktop	4.87.0	2026-08-17 release
WSL	WSL2 ≥ 2.1.5	Docker minimum

Suricata 8.0.6은 2026년 7월 Security Release이며 Suricata 7은 EOL이다.

Snort 공식 Download Page는 3.12.2.0을 제공한다.

Wazuh Docker 공식 예시는 현재 **v4.14.7** branch를 사용한다.

OpenSearch 공식 최신 Documentation은 3.8이다.

Zeek 8.2는 2026년 5월 공개되었으며 Zeek 9.0은 2026년 8월 31일 출시 예정이므로 **2026년 8월 24일 기준 Baseline에는 포함하지 않는다.**

MITRE ATT&CK Content Version은 19.2이다.

Docker Desktop 공식 Release Notes상 최신 공개 Release는 2026년 8월 17일의 4.87.0이다.

67. OpenSearch와 Zeek의 현재 위치

OpenSearch

P2 또는 Wazuh Indexer Architecture 학습 시 참조한다.

현재 OpenSearch Documentation은 3.8을 제공하며 Search/Analytics 및 Dashboards를 지원한다.

본 MVP에서는 별도 OpenSearch Cluster를 추가하지 않는다.

Zeek

P2 Network Security Monitoring 확장 Component이다.

```
Mirrored Traffic
↓
Zeek
↓
conn.log
dns.log
http.log
...
```

현재 Architecture에는 미포함한다.

68. Architecture Risk

Risk	가능성	영향	Mitigation
Port Mirror 오설정	High	High	AV-002 우선 검증
Suricata Interface 오류	Medium	High	Capture Health Check
WSL Networking 혼동	Medium	Medium	IDS는 VM으로 분리

Risk	가능성	영향	Mitigation
Wazuh RAM 부족	High	High	8GB 이상 확보
Host RAM 부족	Medium	High	단계별 Startup
PCAP Storage 증가	High	Medium	Retention
EVE Log 증가	High	Medium	Rotation
Duplicate Alert	Medium	Medium	Snort Offline 기본
False Positive	High	Medium	Tuning Workflow
Attack 외부 유출	Low	Critical	Private vSwitch
Git Secret 노출	Medium	Critical	Secret Scan
Version 불일치	Medium	Medium	Version Pinning
Dashboard 외부 노출	Low	High	MGMT-only Binding

69. Architecture 핵심 Risk — Traffic Visibility

본 프로젝트에서 가장 큰 구조적 Risk는

|"Suricata는 정상 실행되지만 실제 공격 Traffic이 Sensor에 전달되지 않는 상황"

이다.

따라서 구현 순서를 반드시

```

Connectivity
↓
Packet Capture
↓
Port Mirror
↓
Suricata
↓
Rule

```

순으로 진행한다.

Suricata 설치 성공을 IDS 구축 성공으로 판단하지 않는다.

70. Architecture 핵심 Risk — Wazuh 과투자

Wazuh부터 구축하지 않는다.

잘못된 순서:

```
Wazuh
→ Dashboard
→ Suricata
→ Network
```

권장 순서:

```
Network
→ Mirror
→ Packet
→ Suricata
→ EVE
→ Wazuh
```

이렇게 해야

왜 Alert가 안 뜨는가?

라는 문제를 계층별로 진단할 수 있다.

71. Architecture Baseline v1.0

항목	최종 결정
Deployment Model	Hybrid
Host	Windows
Hypervisor	Hyper-V
Container Runtime	Docker Desktop / WSL2
Attacker	Linux VM
Gateway	Linux VM
Victim	Linux VM
Primary IDS	Suricata 8.0.6
Secondary IDS	Snort 3.12.2.0
Capture Method	Hyper-V Port Mirroring
IDS Capture	AF_PACKET

항목	최종 결정
Offline Validation	PCAP
SIEM	Wazuh 4.14.7 Single-node
Packet Analysis	Wireshark / tshark
Management Network	10.77.10.0/24
Attack Network	10.77.20.0/24
Victim Network	10.77.30.0/24
Sensor Monitor NIC	No IP
Log Pipeline	Suricata → EVE → Wazuh Agent → Wazuh
Primary Evidence	EVE + PCAP + Rule + Screenshot
Rule Repository	Git
ATT&CK	Content 19.2
Dashboard	MGMT only
Internet	Attack Lab 기본 차단

72. Architecture Baseline Diagram



73. 상세설계(LLD) 이관사항

다음은 HLD에서 결정하지 않고 LLD에서 실제 값으로 확정한다.

VM

- VM Name
- OS Version
- VM Generation
- CPU
- RAM
- Disk

Network

- vSwitch Name
- Exact IP
- MAC
- Interface Name
- Route
- Gateway

Port Mirror

- Source Adapter
- Destination Adapter
- Hyper-V Configuration

Suricata

- Interface
- `suricata.yaml`
- HOME_NET
- Rule Path
- SID Range
- EVE Event Types
- Rotation

Snort

- Lua Config
- Rule Directory
- PCAP Command
- Output Format

Wazuh

- Docker Compose

- Volume
- Port Binding
- Agent Configuration
- Certificate
- User
- Password Policy

Firewall

- Source
- Destination
- Port
- Direction
- Action

Testing

- Command
- Expected Result
- Evidence

74. LLD에서 반드시 검증할 항목

1. Hyper-V 활성화
2. Virtual Switch 생성
3. VM Network Adapter
4. Zone별 Routing
5. Attack→MGMT 차단
6. Port Mirror
7. Sensor Interface
8. AF_PACKET
9. Suricata EVE
10. Snort PCAP
11. Wazuh Agent
12. Wazuh Manager
13. Indexer
14. Dashboard
15. Port Binding
16. Storage Path

- 17. Log Rotation
- 18. Evidence Naming
- 19. Health Check
- 20. Backup

75. 구현 순서

본 Architecture를 기준으로 실제 구현은 다음 순서로 진행한다.

```
Phase 0
Host / Hyper-V Readiness

    ↓

Phase 1
Virtual Network

    ↓

Phase 2
Attacker / Gateway / Victim

    ↓

Phase 3
Sensor Traffic Visibility

    ↓

Phase 4
Suricata

    ↓

Phase 5
PCAP / Snort Validation

    ↓

Phase 6
Wazuh Single-node

    ↓

Phase 7
Suricata → Wazuh

    ↓

Phase 8
Attack Scenarios
```

↓
Phase 9
SOC Investigation

↓
Phase 10
Detection Tuning

↓
Phase 11
Portfolio Evidence

76. 최종 Architecture 완료조건

HLD 자체의 완료조건은 다음과 같다.

- ☑ ~~Deployment Model 선정~~
- ☑ ~~Hypervisor 선정~~
- ☑ ~~IDS Primary/Secondary 역할 구분~~
- ☑ ~~Sensor Placement 확정~~
- ☑ ~~Capture 방식 확정~~
- ☑ ~~Network Zone 확정~~
- ☑ ~~CIDR Baseline 확정~~
- ☑ ~~Traffic Path 정의~~
- ☑ ~~Wazuh Architecture 정의~~
- ☑ ~~Log Pipeline 정의~~
- ☑ ~~Source of Truth 정의~~
- ☑ ~~Storage 구조 정의~~
- ☑ ~~Security Boundary 정의~~
- ☑ ~~Management/Data Plane 분리~~
- ☑ ~~Port Matrix 정의~~
- ☑ ~~Resource Baseline 정의~~
- ☑ ~~Failure Domain 정의~~

- ✓ Evidence Architecture 정의
- ✓ ADR 작성
- ✓ Requirements Traceability 정의
- ✓ Architecture Validation 정의
- ✓ MVP Architecture 정의
- ✓ Portfolio Demonstration Architecture 정의
- ✓ Technology Version 검증
- ✓ Risk 분석
- ✓ LLD 어관사항 정의

77. Architecture 최종 판단

본 HLD의 핵심은 다음과 같다.

Suricata를 설치하는 것이 아니라 Suricata가 관찰해야 할 Network Traffic을 먼저 설계한다.

이를 위해

Attacker
→ Gateway
→ Victim

이라는 실제 Traffic Path를 만들고,
Victim Traffic을

Hyper-V Port Mirroring

으로 복제하여

Security Sensor
→ Suricata

에 전달한다.

Suricata의 Detection Event는

```
eve.json
→ Wazuh Agent
→ Wazuh Server
→ Indexer
→ Dashboard
```

로 전달된다.

SOC Analyst는 여기에서 끝내지 않고

```
Alert
→ Raw EVE
→ PCAP
→ Rule
→ ATT&CK
→ Verdict
```

를 수행한다.

False Positive라면

```
Rule Tuning
→ Git
→ Re-test
```

과정까지 진행한다.

78. Portfolio 핵심 설명문

면접에서는 본 Architecture를 다음과 같이 설명할 수 있어야 한다.

“Hyper-V를 이용해 공격자, 피해자, 보안센서를 논리적으로 분리하고, Victim Traffic을 Virtual Switch Port Mirroring으로 Suricata Sensor에 복제하도록 설계했습니다. Suricata는 Passive IDS로 동작하면서 AF_PACKET을 통해 Traffic을 분석하고 EVE JSON을 생성합니다. EVE Event는 Wazuh Agent를 통해 Wazuh SIEM으로 전달되며, Alert가 발생하면 PCAP·Raw Event·Signature Rule을 함께 분석하여 True Positive 또는 False Positive를 판별합니다. 오탐인 경우 Detection Rule을 수정하고 동일 Traffic으로 재검증하며 변경이력과 결과를 Git 및 Evidence로 남기는 구조입니다.”

이 문장이 본 Portfolio Architecture의 핵심이다.

79. 다음 산출물

본 HLD 완료 후 다음 산출물은

「보안관제 프로젝트 상세설계서(LLD) v1.0」

으로 한다.

LLD에서는 본 HLD의 Architecture Baseline을 변경하지 않고 다음을 실제 구현값까지 구체화한다.

```
Hyper-V VM
  ↓
vSwitch
  ↓
CIDR / IP
  ↓
Routing / Firewall
  ↓
Port Mirroring
  ↓
Sensor Interface
  ↓
Suricata Configuration
  ↓
Snort Configuration
  ↓
Docker Network
  ↓
Wazuh Configuration
  ↓
Port / Volume
  ↓
Health Check
  ↓
Test Case
  ↓
Evidence
```

문서 진행상태

```
Requirements Definition
  |
  | COMPLETE
  ▼
HLD / System Architecture
  |
  | ← CURRENT BASELINE COMPLETE
  ▼
```


LLD / Detailed Design



Implementation Plan



Infrastructure Build



IDS Deployment



SIEM Integration



Attack Simulation



Detection Engineering



SOC Investigation



Validation



Portfolio Completion

Architecture Baseline v1.0 — FINAL

최종 채택 Architecture

Hyper-V 기반의 격리된 Attack/Victim Network에서 발생한 Traffic을 Virtual Switch Port Mirroring을 통해 Linux Security Sensor로 복제하고, Suricata 8을 Primary Passive IDS로 사용하여 탐지한다. 탐지 Event는 EVE JSON으로 보존하고 Wazuh Agent를 통해 Docker 기반 Wazuh Single-node SIEM으로 전달한다. Snort 3는 동일 PCAP에 대한 Secondary Validation Engine으로 사용한다. SOC Analyst는 Wazuh Alert, Raw EVE, PCAP, Detection Rule 및 MITRE ATT&CK을 연결하여 사고를 분석하며 False Positive 발생 시 Rule Tuning과 Re-test까지 수행하고 모든 결과를 Git 및 Evidence Repository에 남긴다.

Architecture Status: **BASELINE APPROVED FOR LLD**